



Nexa Caplogy

Table des matières



```
[12pt]article [utf8]inputenc [french]babel graphicx listings xcolor tikz enumitem tcolorbox geometry
margin=1in
```

10 Challenges Simples sur les Arbres en Data Science

Applications pratiques pour débutants en données

Résumé

10 challenges pratiques sur les arbres appliqués à la data science. Chaque challenge est simple, concret et utile pour comprendre comment les arbres sont utilisés dans le traitement des données.

Table des matières

Challenges de Base en Data Science

Challenge 1 : Organiser des données client

Challenge Data Science

Contexte : Vous avez des données clients et vous voulez les organiser hiérarchiquement pour une analyse marketing.

Données :

```
1 clients = [  
2     {"id": 1, "age": 25, "ville": "Paris", "departement": "75"},  
3     {"id": 2, "age": 35, "ville": "Lyon", "departement": "69"},  
4     {"id": 3, "age": 28, "ville": "Paris", "departement": "75"},  
5     {"id": 4, "age": 42, "ville": "Marseille", "departement": "13"},  
6     {"id": 5, "age": 31, "ville": "Lyon", "departement": "69"},  
7 ]
```

Objectif : Créer un arbre qui organise les clients par : 1. Département 2. Ville 3. Groupe d'âge (<30, 30-40, >40)

Structure attendue :

France

75 (Paris)

<30 ans

Client 1, Client 3

30-40 ans

69 (Lyon)

30-40 ans

Client 2, Client 5

13 (Marseille)

>40 ans

Client 4

Exercice :

1. Créez la structure de l'arbre
2. Ajoutez les clients aux bonnes branches
3. Calculez le nombre de clients par niveau

Challenge 2 : Arbre de décision simple

Challenge Data Science

Contexte : Prédire si un client va acheter un produit basé sur son âge et sa ville.

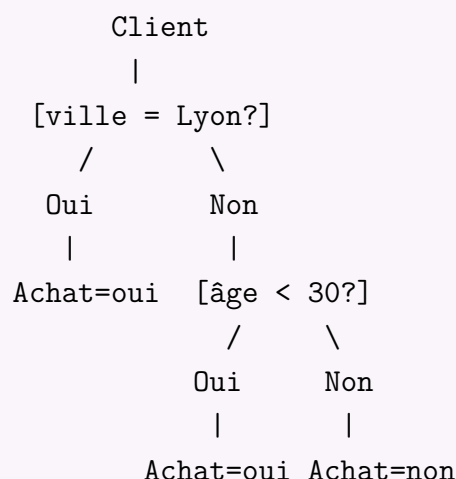
Données d'entraînement :

```
1 data = [
2     {"age": 25, "ville": "Paris", "achat": "oui"},
3     {"age": 35, "ville": "Lyon", "achat": "oui"},
4     {"age": 45, "ville": "Paris", "achat": "non"},
5     {"age": 28, "ville": "Marseille", "achat": "non"},
6     {"age": 31, "ville": "Lyon", "achat": "oui"},
7     {"age": 40, "ville": "Paris", "achat": "non"}
8 ]
```

Objectif : Construire manuellement un arbre de décision avec ces règles :

- Si ville = "Lyon" → Achat = "oui"
- Sinon, si âge < 30 → Achat = "oui"
- Sinon → Achat = "non"

Arbre attendu :



Exercice :

1. Implémentez l'arbre de décision
2. Testez avec de nouveaux clients :
 - Client : âge=27, ville="Paris"
 - Client : âge=38, ville="Marseille"
3. Calculez l'accuracy sur les données d'entraînement

Challenge 3 : Hiérarchie produit-catégorie

Challenge Data Science

Contexte : Gérer le catalogue produits d'un e-commerce.

Données :

```
1 produits = [  
2     {"id": "P001", "nom": "iPhone 13", "cat_gorie": "Smartphone", "  
3       sous_cat_gorie": "Apple"},  
4     {"id": "P002", "nom": "Galaxy S21", "cat_gorie": "Smartphone", "  
5       sous_cat_gorie": "Samsung"},  
6     {"id": "P003", "nom": "MacBook Air", "cat_gorie": "Ordinateur", "  
7       sous_cat_gorie": "Apple"},  
8     {"id": "P004", "nom": "ThinkPad", "cat_gorie": "Ordinateur", "  
9       sous_cat_gorie": "Lenovo"},  
10    {"id": "P005", "nom": "AirPods", "cat_gorie": "Audio", "  
11      sous_cat_gorie": "Apple"}  
12 ]
```

Objectif : Créer un arbre pour naviguer dans les produits :

Catalogue

Électronique

Smartphone

Apple

iPhone 13

Samsung

Galaxy S21

Ordinateur

Apple

MacBook Air

Lenovo

ThinkPad

Audio

Apple

AirPods

Exercice :

1. Créez la structure de l'arbre
2. Implémentez une fonction pour chercher tous les produits d'une marque
3. Comptez le nombre de produits par catégorie
4. Trouvez toutes les catégories d'Apple

Challenges d'Analyse de Données

Challenge 4 : Segmentation clients avec arbre

Challenge Data Science

Contexte : Segmenter les clients selon leur comportement d'achat.

Données de ventes :

```
1 ventes = [  
2     {"client": "Alice", "montant": 150, "fr quence": 3, "segment": "A"},  
3     {"client": "Bob", "montant": 80, "fr quence": 5, "segment": "B"},  
4     {"client": "Charlie", "montant": 200, "fr quence": 2, "segment": "A"  
5     },  
6     {"client": "Diana", "montant": 50, "fr quence": 1, "segment": "C"},  
7     {"client": "Eve", "montant": 120, "fr quence": 4, "segment": "B"}  
8 ]
```

Règles de segmentation :

- Si montant > 100 ET fréquence > 2 → Segment A (Clients VIP)
- Sinon, si montant > 100 OU fréquence > 2 → Segment B (Clients réguliers)
- Sinon → Segment C (Clients occasionnels)

Objectif : Construire un arbre de décision qui applique ces règles.

Exercice :

1. Créez l'arbre de décision
2. Classez ces nouveaux clients :
 - Client : montant=90, fréquence=4
 - Client : montant=110, fréquence=1
 - Client : montant=60, fréquence=2
3. Calculez la répartition des segments

Challenge 5 : Arbre pour analyse de sentiment

Challenge Data Science

Contexte : Classifier des tweets comme positifs, neutres ou négatifs.

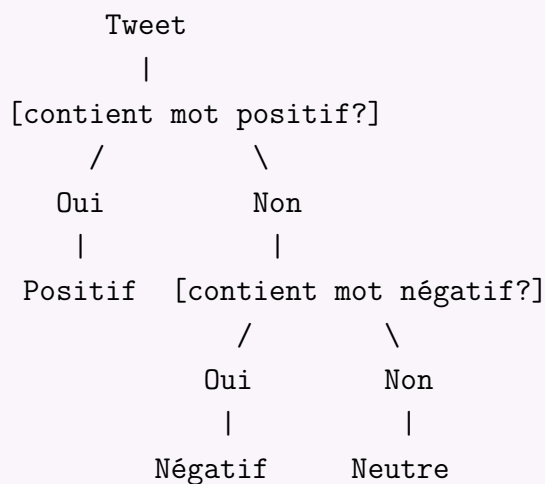
Données d'entraînement :

```
1 tweets = [
2     {"texte": "J'adore ce produit !", "sentiment": "positif"},
3     {"texte": "Service horrible", "sentiment": "n gatif"},
4     {"texte": "Pas mal", "sentiment": "neutre"},
5     {"texte": "Incroyable exp rience", "sentiment": "positif"},
6     {"texte": "Je d teste", "sentiment": "n gatif"},
7     {"texte": "Correct", "sentiment": "neutre"}
8 ]
```

Objectif : Créer un arbre simple basé sur des mots-clés :

- Si contient "adore" ou "incroyable" → positif
- Sinon, si contient "horrible" ou "déteste" → négatif
- Sinon → neutre

Arbre attendu :



Exercice :

1. Implémentez l'arbre de classification
2. Testez avec :
 - "Ce produit est incroyable !"
 - "Service moyen"
 - "Je déteste attendre"
3. Calculez la précision

Challenge 6 : Organisation de données géographiques

Challenge Data Science

Contexte : Organiser des magasins par localisation géographique.

Données magasins :

```

1 magasins = [
2     {"nom": "Paris Centre", "ville": "Paris", "r_gion": "Île-de-France",
3       "chiffre_affaires": 100000},
4     {"nom": "Lyon Part-Dieu", "ville": "Lyon", "r_gion": "Auvergne-Rhône-Alpes", "chiffre_affaires": 80000},
5     {"nom": "Marseille Vieux Port", "ville": "Marseille", "r_gion": "Provence-Alpes-Côte d'Azur", "chiffre_affaires": 70000},
6     {"nom": "Paris Montparnasse", "ville": "Paris", "r_gion": "Île-de-France", "chiffre_affaires": 120000},
7     {"nom": "Lyon Presqu'île", "ville": "Lyon", "r_gion": "Auvergne-Rhône-Alpes", "chiffre_affaires": 90000}
8 ]

```

Objectif : Créer un arbre hiérarchique :

```

France
  Île-de-France
    Paris
      Paris Centre (CA: 100k)
      Paris Montparnasse (CA: 120k)
  Auvergne-Rhône-Alpes
    Lyon
      Lyon Part-Dieu (CA: 80k)
      Lyon Presqu'île (CA: 90k)
  Provence-Alpes-Côte d'Azur
    Marseille
      Marseille Vieux Port (CA: 70k)

```

Exercice :

1. Créez l'arbre hiérarchique
2. Calculez le chiffre d'affaires total par région
3. Trouvez la ville avec le plus de magasins
4. Ajoutez un nouveau magasin : "Nice Promenade", ville : "Nice", région : "PACA", CA : 60000

Challenges de Visualisation

Challenge 7 : Arbre pour analyse de prix

Challenge Data Science

Contexte : Analyser des produits par fourchette de prix.

Données produits :

```
1 produits = [  
2     {"nom": "Caf ", "cat_gorie": "Boisson", "prix": 5},  
3     {"nom": "Th ", "cat_gorie": "Boisson", "prix": 4},  
4     {"nom": "Croissant", "cat_gorie": "Pâtisserie", "prix": 2},  
5     {"nom": "Pain au chocolat", "cat_gorie": "Pâtisserie", "prix": 3},  
6     {"nom": "Sandwich", "cat_gorie": "Plat", "prix": 8},  
7     {"nom": "Salade", "cat_gorie": "Plat", "prix": 10}  
8 ]
```

Objectif : Créer un arbre qui classe les produits par : 1. Catégorie 2. Fourchette de prix (€ : <3, 3-5, >5)

Structure attendue :

Menu

Boisson

<3€

3-5€

Café (5€)

Thé (4€)

>5€

Pâtisserie

<3€

Croissant (2€)

Pain au chocolat (3€)

3-5€

>5€

Plat

<3€

3-5€

>5€

Sandwich (8€)

Salade (10€)

Exercice :

1. Construisez l'arbre
2. Comptez le nombre de produits par fourchette de prix
3. Trouvez la catégorie avec le prix moyen le plus élevé
4. Ajoutez un nouveau produit : "Smoothie", catégorie : "Boisson", prix : 6

Challenge 8 : Arbre de décision pour recommandation

Challenge Data Science

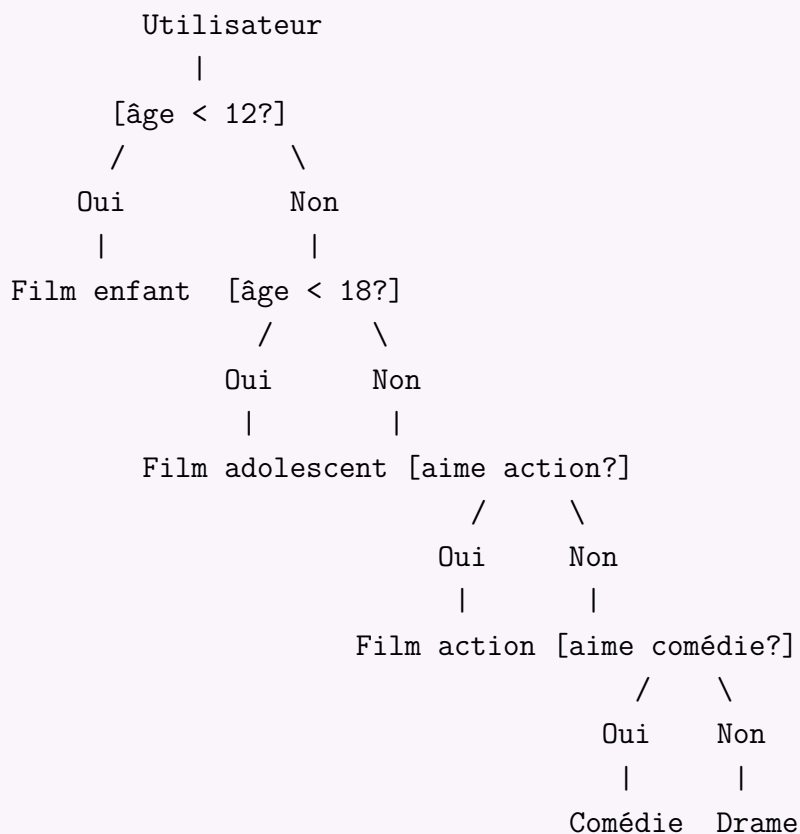
Contexte : Recommander des films basé sur l'âge et les préférences.

Règles de recommandation :

- Si âge < 12 → Film enfant
- Sinon, si âge < 18 → Film adolescent
- Sinon :
 - Si aime action → Film d'action
 - Si aime comédie → Comédie
 - Sinon → Drame

Objectif : Créer un arbre de décision pour ces règles.

Arbre complet :



Exercice :

1. Implémentez l'arbre de décision
2. Testez avec :
 - Âge : 10, préférence : action
 - Âge : 16, préférence : comédie
 - Âge : 25, préférence : action
 - Âge : 30, préférence : romance
3. Ajoutez une nouvelle règle : si âge > 60 → Film classique

Challenges d'Optimisation

Challenge 9 : Recherche efficace dans un catalogue

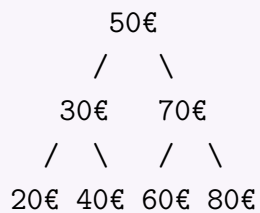
Challenge Data Science

Contexte : Rechercher rapidement des produits dans un grand catalogue.

Données : 1000 produits avec catégories et prix

Objectif : Créer un arbre binaire de recherche pour trouver rapidement les produits par prix.

Structure BST pour les prix :



Exercice :

1. Créez un BST avec ces prix : [50, 30, 70, 20, 40, 60, 80]
2. Implémentez la recherche d'un produit par prix
3. Trouvez tous les produits entre 35€ et 65€
4. Calculez le temps de recherche moyen (théorique)

Challenge 10 : Analyse de performance employés

Challenge Data Science

Contexte : Évaluer les employés selon leur performance.

Données :

```
1 employes = [  
2     {"nom": "Alice", "d partement": "Ventes", "performance": 85},  
3     {"nom": "Bob", "d partement": "Tech", "performance": 92},  
4     {"nom": "Charlie", "d partement": "Ventes", "performance": 78},  
5     {"nom": "Diana", "d partement": "Marketing", "performance": 88},  
6     {"nom": "Eve", "d partement": "Tech", "performance": 95}  
7 ]
```

Critères d'évaluation :

- Performance > 90 → Excellent
- Performance 80-90 → Bon
- Performance < 80 → À améliorer

Objectif : Créer un arbre hiérarchique :

Entreprise

Ventes

Excellent

Bon

Alice (85)

À améliorer

Charlie (78)

Tech

Excellent

Bob (92)

Eve (95)

Bon

À améliorer

Marketing

Excellent

Bon

Diana (88)

À améliorer

Exercice :

1. Construisez l'arbre
2. Calculez la performance moyenne par département
3. Trouvez le département avec la meilleure performance moyenne
4. Identifiez les employés "Excellent"

Guide de Réalisation

Conseils pour tous les challenges

- **Commencez simple** : Implémentez d'abord une version basique
- **Testez avec peu de données** : Vérifiez que ça marche avant d'ajouter plus
- **Utilisez des classes** : Pour représenter les nœuds de l'arbre
- **Ajoutez des méthodes** : Pour ajouter, chercher et compter

Structure de base recommandée

```
1 class Noeud:
2     def __init__(self, valeur):
3         self.valeur = valeur
4         self.enfants = [] # Pour les arbres n-aires
5         # ou
6         self.gauche = None # Pour les arbres binaires
7         self.droite = None
8
9 class Arbre:
10     def __init__(self):
11         self.racine = None
12
13     def ajouter(self, valeur, parent=None):
14         # Implémentation à compléter
15         pass
16
17     def chercher(self, valeur):
18         # Implémentation à compléter
19         pass
20
21     def afficher(self):
22         # Implémentation à compléter
23         pass
```

Solutions partielles suggérées

Pour chaque challenge, pensez à : 1. Définir la structure de données 2. Implémenter l'ajout d'éléments 3. Implémenter la recherche 4. Ajouter des statistiques (comptage, moyenne, etc.) 5. Tester avec les exemples fournis

Applications réelles

Ces challenges simulent des cas réels :

- Organisation de données clients (CRM)
- Classification (machine learning simple)
- Gestion de catalogue (e-commerce)

- Analyse de performance (business intelligence)
- Systèmes de recommandation

Prochaines étapes

Après avoir complété ces challenges :

1. Ajoutez la persistance (sauvegarde dans un fichier)
2. Implémentez la suppression d'éléments
3. Ajoutez des statistiques avancées
4. Créez une interface simple (avec print)
5. Exportez les résultats en CSV